

Search and A*: An End-to-End Explainer

Warehouse Robot Navigation — CS F407 Artificial Intelligence

Study notes, not for submission

How to read this

This is the whole laboratory explained from first principles: what a search problem is, how BFS and A* differ, what actually happens inside the code, and why the experiments came out the way they did. Every number quoted here was measured from the programs in this folder — nothing is illustrative.

If you only read one section, read §6. It is the part of this laboratory that teaches something you cannot get from a textbook.

Contents

1	The problem	1
2	Turning a maze into a search problem	1
2.1	State versus node — the idea everything depends on	2
2.2	Why a tuple	2
3	The family of search algorithms	2
3.1	Blind versus informed	3
4	$f = g + h$	3
4.1	The heuristic used here	3
4.2	Worked example on the real map	3
5	What the code actually does	4
5.1	The three data structures	4
5.2	Why a heap	4
5.3	Why there is a counter in the tuple	4
5.4	Lazy deletion — and why “generated” exceeds “expanded”	4
5.5	Testing for the goal on <i>pop</i> , not on generation	4
6	The bug that taught the lesson	5
6.1	What happened	5
6.2	Why	5
6.3	The fix	5
6.4	The result, drawn from real traces	5
6.5	The lesson	6
7	Why the supplied warehouse hides all of this	6
8	Three properties of a heuristic	6
8.1	Admissible: never overestimates	7
8.2	Consistent: never jumps	7
8.3	Informative: how close to the truth	7

9 Breaking admissibility on purpose	7
9.1 Why overestimating breaks it	8
10 What to remember	8
11 Questions worth being able to answer	8

1 The problem

A robot sits in a warehouse laid out as a grid. It starts at **S**, must reach **G**, and shelves (**#**) block the way. It moves one cell at a time — up, down, left or right — and every move costs 1.

```
#####
#S....#.....#      S = start  (row 1, column 1)
#..###.#.#####.#   G = goal   (row 7, column 15)
#...#.#.....#.#     # = shelf
###.#.#####.#.#    . = free floor
#...#.....#.#
#.#####.#.#         9 rows x 17 columns
#.....#G#           64 free cells
#####
```

The answer, which the program finds, is a 40-move route. But the point of the laboratory is not the number 40. It is *how* you get a machine to find it, and how you convince yourself the machine is right.

2 Turning a maze into a search problem

Search is a general method. Before you can apply it, you have to say what the problem *is* in the vocabulary the method uses. That vocabulary has six parts:

$$\mathcal{P} = (S, A, T, s_0, G, c)$$

Symbol	Name	For this warehouse
S	states	every free cell, as a pair (row, col) — 64 of them
A	actions	Up, Down, Left, Right
T	transition	move one cell, if the target is in bounds and not a shelf
s_0	initial state	(1, 1)
G	goal states	{(7, 15)}
c	cost	1 per move

Most of this is obvious. One part is not, and it is the part that matters.

2.1 State versus node — the idea everything depends on

A **state** is a configuration of the world. A **node** is a state *plus* the bookkeeping the search is carrying about how it got there.

For this problem the state is just the cell, (row, col). It is tempting to also put “the route I took to get here” or “how many steps I have used” into the state. Do not. Those are *history*, and they belong to the node.

Why it matters: suppose two different routes both arrive at cell (5,7), one having used 12 moves and one having used 18. From (5,7) onward, the remaining problem is *identical* in both

cases — the same walls, the same goal, the same choices. That is what lets the search say “I have already been here, and I got here more cheaply last time, so I can discard this.”

If you folded the route into the state, those two arrivals would be two *different* states, nothing would ever look like a repeat, and the search would degenerate into enumerating every possible walk through the maze — which is astronomically many.

The test for what belongs in a state

Ask: *if I told you only this, could you work out what the robot can do next and what it will cost?* If yes, it is state. If it only tells you about the past, it is history — keep it on the node.

So in the code:

- **state** is (r, c) — a plain tuple.
- **g**, the parent pointer, and **f** live in dictionaries keyed by state. They are node information.

2.2 Why a tuple

(r, c) is a tuple, not a list, for two concrete reasons: tuples are **hashable**, so they can be dictionary keys and set members — which is exactly what “have I seen this state before?” needs; and they are **immutable**, so a state cannot be accidentally changed after being stored as a key.

3 The family of search algorithms

Every algorithm in this family shares one loop:

1. Take a state out of the **frontier** (the set of places you know about but have not explored from yet).
2. If it is the goal, stop.
3. Otherwise **expand** it: generate its legal neighbours and put them in the frontier.

The only thing that differs between these algorithms

They differ in *one* decision: which state to take out of the frontier next. That single choice is what separates BFS from DFS from A*. Everything else is the same loop.

Algorithm	Picks next	Frontier	Optimal?
BFS	shallowest (fewest moves)	FIFO queue	yes, if all costs equal
DFS	deepest	LIFO stack	no
Uniform-cost	cheapest so far, g	priority queue	yes
Greedy	looks closest to goal, h	priority queue	no
A*	best total estimate, $g + h$	priority queue	yes, if h admissible

3.1 Blind versus informed

BFS, DFS and uniform-cost are **blind** (or uninformed): they know where they have been, but nothing about where the goal is. BFS expands in a growing ring around the start, treating the direction of the goal as irrelevant.

A* is **informed**: it is given a function h that estimates the cost still remaining. That is the entire difference, and the word “informed” means exactly this and nothing more.

4 $f = g + h$

A* orders its frontier by

$$f(n) = g(n) + h(n)$$

where

- $g(n)$ = cost actually paid to reach n from the start — *known*,
- $h(n)$ = estimated cost from n to the goal — *guessed*,
- $f(n)$ = estimated total cost of a solution passing through n .

So A* always works on the node that currently looks like it lies on the best complete solution. Note the shape of this: g looks backwards and is a fact; h looks forwards and is an estimate. A* is the algorithm that adds a fact to a guess and sorts by the sum.

4.1 The heuristic used here

$$h((r, c)) = |r - r_G| + |c - c_G| \quad (\text{Manhattan distance})$$

Named after the street grid: you cannot cut diagonally through a block, so the distance is the sum of the horizontal and vertical legs.

Where does it come from? **Relax the problem.** Take the real warehouse and delete every shelf. In that easier world, the cheapest route from (r, c) to the goal is exactly $|r - r_G| + |c - c_G|$ moves — you close the row gap and the column gap, and no move closes both at once because diagonals are not allowed.

Where good heuristics come from

Solve an easier version of your problem exactly, and use that answer as the estimate. Because you removed constraints, the easier answer can never be worse than the real one — which is precisely the property A* needs.

4.2 Worked example on the real map

Start is $(1, 1)$, goal is $(7, 15)$.

$$h(s_0) = |1 - 7| + |1 - 15| = 6 + 14 = 20$$

So **no route can be shorter than 20 moves**. The program returns 40 — the shelves double the ideal distance. That 20 is a genuinely useful check: if the program had ever returned 18, it would be wrong without needing to look at the path.

Take a cell partway along, say $(5, 9)$, reached in $g = 12$ moves:

$$h = |5 - 7| + |9 - 15| = 2 + 6 = 8, \quad f = 12 + 8 = 20.$$

5 What the code actually does

5.1 The three data structures

Name	Type	Holds
frontier	min-heap	states to explore, ordered by f
best_g	dict	cheapest known cost to each state
parent	dict	which state we came from
closed	set	states already expanded

5.2 Why a heap

You need “give me the smallest f ” repeatedly. Scanning a list is $O(n)$ every time. A binary heap gives you the minimum in $O(\log n)$. Python’s `heapq` compares tuples left to right, so pushing $(f, \dots, \text{state})$ sorts by f automatically.

5.3 Why there is a counter in the tuple

Entries are pushed as $(f, -g, \text{counter}, \text{state})$. The counter exists because if f and $-g$ both tie, Python moves on to the next element and would start comparing the *state tuples* — silently ordering the search by geography, which is meaningless. A unique, increasing counter guarantees the comparison always stops there.

5.4 Lazy deletion — and why “generated” exceeds “expanded”

If we find a cheaper route to a state already sitting in the frontier, we ought to lower its priority. Python’s heap has no such operation. So instead we push a *second* entry for that state with the better f , and let both sit there.

The better one is nearer the top, so it comes out first. When the stale one eventually surfaces, the state is already in `closed` and we skip it.

Reading the statistics

Generated counts pushes onto the frontier. **Expanded** counts states actually taken out and explored. Generated is always $\geq$ expanded, because some pushes are stale duplicates that get skipped. This is not a bug — it is what lazy deletion looks like from the outside.

5.5 Testing for the goal on *pop*, not on generation

This is the subtlest correctness point in the whole program.

When you generate a neighbour and notice it is the goal, it is tempting to return immediately. **Do not.** At that moment you have found *a* route to the goal, not necessarily the *cheapest* one — a better route may still be sitting in the frontier, unexplored.

A* is only guaranteed optimal if you wait until the goal is *popped*. When the goal comes off the top of the heap, its f is the smallest in the frontier, and since $h(\text{goal}) = 0$, that f is its true cost g . Nothing cheaper remains. Only then is it safe to stop.

6 The bug that taught the lesson

This is the important section.

6.1 What happened

The generated A* was correct. It passed all 20 tests. It returned optimal paths on every map. And then this measurement appeared, on an open test map with 130 free cells:

	Path length	States expanded	Peak frontier
BFS	25	116	8
A* with Manhattan	25	116	8

Identical. Not similar — *identical*, including the peak frontier size. An informed search does not coincidentally match a blind one to the digit. The heuristic was being computed correctly and then having no effect whatsoever.

6.2 Why

Consider any cell on a sensible, detour-free route from start to goal. Every move along such a route increases g by 1 and decreases h by 1. So the sum does not move:

$$f = g + h = \text{constant} = h(s_0).$$

Checking the open map directly: **all 130 free cells have $f = 25$** . There is no ordering information in f at all. It is one enormous plateau.

The failure

When every entry ties on f , the heap's ordering is decided entirely by the tie-break. The tie-break was first-in-first-out. FIFO order over a frontier *is* breadth-first order. So A* was running BFS — correctly, optimally, and pointlessly.

6.3 The fix

Break ties toward the **larger** g — the node further along a route. Push $(f, -g, \text{counter}, \text{state})$; the heap wants small values, and $-g$ is smallest when g is largest.

Intuition: among many nodes that all claim the same total cost 25, prefer the one that has already *paid* 22 of it over one that has paid 3. The first is nearly at the goal; the second has all the work still ahead. Ties should be resolved toward finishing, not toward fanning out.

6.4 The result, drawn from real traces

Below, * marks a cell the algorithm actually expanded and . marks one it never touched. These are recorded traces, not sketches.

BFS — 116 of 130 cells (89% of the map)

```
#####
#S#####
#####
#####
#####.....#
#####.....#
#####
#####
#####G#
#####
```

A* with the tie-break fixed — 26 of 130 cells (20%)

```
#####
#S.....#
#*.....#
#*...#####.....#
#*...#.....#.....#
#*...#.....#.....#
#*...#####.....#
#*.....#
#####G#
#####
```

BFS floods the room. A* walks down the left wall and along the bottom — almost exactly the path, and nearly nothing else. Same 25-move answer, 78% less work.

6.5 The lesson

Working output $\neq$ validated algorithm

No test caught this. Every path returned was legal and optimal. The defect was found because a *number was implausible* — and it was only implausible because there was an expectation to compare it against.

If you accept generated code because its output looks right, you inherit exactly this class of defect: silently inert machinery, invisible precisely where the code is most sophisticated.

Note also where the fault really lay. The design specified the frontier completely except for the tie-break rule, which it left vague. The LLM implemented what was specified and chose the obvious default for what was not. **It was a good engineer working to bad drawings.**

7 Why the supplied warehouse hides all of this

On the warehouse itself, BFS and A* both expand all 64 cells. A* saves nothing. This is not a failure — it is a property of the map.

The peak frontier on the warehouse is **3**. That means at almost every step the search had at most three options. The warehouse is a maze of single-width corridors: you walk until you hit a junction, and there are very few junctions.

When a heuristic can help

A heuristic *ranks alternatives*. If there are no alternatives, there is nothing to rank. In a corridor, informed and blind search do exactly the same thing, because there is only one thing to do.

This is why the report added an open-plan control map. Without it, the honest conclusion would have been “A* gave no benefit,” which is true of this map and badly misleading about A*.

8 Three properties of a heuristic

These get confused constantly. They are different things.

8.1 Admissible: never overestimates

$$h(n) \leq h^*(n) \quad \text{for every } n$$

where h^* is the true remaining cost. Manhattan is admissible here: it is the answer to the shelf-free problem, and putting shelves back can only make the real route longer.

Admissibility buys the optimality guarantee. If h is admissible, A* returns a shortest path.

8.2 Consistent: never jumps

$$h(n) \leq c(n, n') + h(n')$$

Each move changes Manhattan distance by at most 1, and each move costs 1 — so it holds. Consistency is stronger than admissibility.

Consistency is what makes the closed set safe. With a consistent h , the first time you expand a state you already have its cheapest route, so you never need to reopen it.

8.3 Informative: how close to the truth

An admissible heuristic can still be useless. $h = 0$ is perfectly admissible and tells you nothing. The closer h gets to h^* from below, the more sharply it discriminates and the fewer states you expand.

Heuristic (open map)	Length	Expanded	
$h = 0$	25	116	admissible, useless
Euclidean	25	92	admissible, weak
Manhattan	25	26	admissible, sharp

All three optimal. A factor of 4.5 between them in work done.

Why is Euclidean weaker? $\sqrt{dr^2 + dc^2} \leq |dr| + |dc|$ always. Euclidean is the true distance only if you could fly diagonally — and you cannot, so it underestimates by more than it needs to. It is a looser lower bound, and looser lower bounds discriminate less.

Admissible and informative are independent

Admissibility is about *correctness*: never overestimate. Informativeness is about *speed*: get as close to the truth as you can. $h = 0$ has the first and none of the second.

9 Breaking admissibility on purpose

What if h *does* overestimate? Try $h = 2 \times$ Manhattan.

On the warehouse and the open map, it returned optimal paths anyway. That is worth understanding: inadmissibility **removes the guarantee; it does not force a violation**. So the experiment was pushed further — a randomised search over maps for a case where it actually breaks. On a cluttered room:

Heuristic	Path length	States expanded
Manhattan	28	133
$2 \times$ Manhattan	30	31

77% less work — and a path two moves worse than optimal. Repeated over 2,754 random solvable maps, doubling the heuristic returned a suboptimal path on **25.9%** of them, worst case 8 moves too long.

9.1 Why overestimating breaks it

A* stops when the goal is popped, reasoning that anything still in the frontier has $f \geq f(\text{goal})$ and therefore cannot be better. That reasoning depends on f being a *lower bound*. If h overestimates, a genuinely cheaper route can be sitting in the frontier carrying an inflated f that makes it look worse than it is. A* never gets to it, and stops with the worse answer.

The trade-off, stated plainly

Inflating the heuristic makes A* more goal-directed and much faster, and pays for it with the optimality guarantee. That is sometimes a good trade — in a game with a millisecond budget, a path 7% too long delivered 4× faster is a clear win. The point is that it must be a *decision*, not an accident.

10 What to remember

1. **State versus node.** State is the cell. History belongs to the node. Getting this wrong breaks duplicate detection and the search explodes.
2. $f = g + h$. A known cost plus an estimated one. g backwards and factual, h forwards and guessed.
3. **Test the goal on pop, not on generation.** Otherwise you return the first route found, not the best one.
4. **Admissible $\Rightarrow$ optimal.** Never overestimate and A* is guaranteed to find a shortest path.
5. **Admissible does not mean useful.** $h = 0$ is admissible and worthless. Closeness to h^* is what buys speed: 116 / 92 / 26.
6. **A* only helps where there are choices.** In a corridor it degrades to BFS, not because it is broken but because there is nothing to rank.
7. **Ties matter as much as the heuristic.** On a unit-cost grid Manhattan makes f constant, and the tie-break rule alone decides whether A* expands 26 states or 116.
8. **Working output is not a validated algorithm.** The bug here produced correct, optimal answers on every test while doing nothing. It was caught by an expectation, not by a test.

11 Questions worth being able to answer

Why is position alone enough to be the state? Because the warehouse is static and the robot carries nothing. Position fully determines what moves are available and what they cost. Everything else is history.

Why test the goal on pop rather than on generation? Generating the goal only proves *a* route exists. When the goal is popped, its f is minimal in the frontier and $h = 0$, so $f = g$ is its true cost and nothing cheaper remains.

Why is Manhattan the right heuristic here? It is the exact solution to the relaxed problem with no shelves. Adding shelves back can only lengthen routes, so it never overestimates — admissible, and also consistent since each unit move changes it by at most 1.

Why did A* expand as many states as BFS on the warehouse? Peak frontier 3 — the map is single-width corridor with almost no junctions. There were no alternatives for the heuristic to rank.

Why can generated exceed expanded? Lazy deletion. An improved route to a state is pushed as a second heap entry rather than updating the existing one, and the stale entry is skipped on pop.

What happens with $h = 0$? A* becomes uniform-cost search, which with unit costs behaves like BFS. Still optimal, maximum work.

What happens if h overestimates? The optimality guarantee is lost. A cheaper route can carry an inflated f , be overlooked, and A* stops on a worse path — measured here at 25.9% of random maps, with a 77% saving in expansions when it happens.

If BFS is also optimal here, why bother with A*? Because BFS is optimal only when every step costs the same, and it does far more work: 116 expansions against 26 on the open map. Change the costs — carpet that slows the robot, say — and BFS stops being optimal at all, while A* with an admissible heuristic still is.

All figures measured from `astar_warehouse.py`, `test_search.py` and `experiments.py`. Verified against an independently written BFS oracle over 6,000 random maps — 36,000 A runs, zero disagreements.*